

AUTH SYSTEM

Complete Developer Documentation

DOCUMENT 1 OF 4

Project Overview, Architecture & Technology Choices

Version	1.0.0
Status	Production Ready
Stack	FastAPI (Python) + Next.js (TypeScript)
Audience	Beginner to Intermediate Developers
Series	4 Documents: Overview, Setup, Security, Deployment

1. What Are We Building?

This documentation covers a complete user authentication system — the part of any web application that handles who you are, how you prove it, and how the system remembers you across sessions. This is the foundation every application needs before building any other feature.

Why This Matters

Authentication is the most security-critical part of any application. A single mistake here can expose every user's data. This system is designed from the ground up with security as the primary concern — not an afterthought.

1.1 Features Included

Registration and account creation:

- User signs up with email, mobile number, username, and password
- Account stays inactive until BOTH email and phone are verified
- Email verification via a secure link sent to the inbox
- Phone verification via a 6-digit SMS one-time password (OTP)

Four login methods after verification:

- Google OAuth — one-click login with a Gmail account
- Email OTP — receive a 6-digit code in your email inbox
- SMS OTP — receive a 6-digit code on your phone

- Username and Password — the traditional login method

Security features built in:

- Rate limiting — automatically blocks brute-force attacks
- HttpOnly cookies — session tokens invisible to JavaScript
- Refresh token rotation — prevents session hijacking
- Audit logging — every security event permanently recorded
- Error notifications — support team alerted on every crash with full context

1.2 The Restaurant Analogy

If you are new to web development, think of this system like a restaurant:

Restaurant Term	Tech Term	What It Does
Customer	Browser / Mobile App	Person using your application
Waiter	Next.js Frontend	Shows pages, collects user input
Kitchen	FastAPI Backend	Processes all business logic
Ingredient Store	PostgreSQL Database	Stores all permanent data
Spice Rack	Redis Cache	Fast temporary storage for OTPs
Delivery Service	Twilio + SendGrid	Sends SMS messages and emails
Secret Recipes	.env Files	Passwords and API keys — never shared

2. Technology Stack — What and Why

Every technology was chosen for a specific reason. This section explains what each one is and why it was selected over alternatives.

2.1 Backend — FastAPI (Python)

Aspect	Detail
What it is	A Python framework that builds APIs — the bridge between database and frontend
Why FastAPI	5x faster than Flask, built-in validation, automatic docs, native async support
Why not Django	Synchronous by default, harder to scale, more complex for pure API services
Async / Await	Handles thousands of simultaneous requests without blocking the server
Pydantic v2	Validates every input automatically — wrong data is rejected before it reaches code

2.2 Frontend — Next.js (TypeScript)

Aspect	Detail
What it is	A React framework that builds the user interface — what people see in the browser
Why Next.js	Server-Side Rendering for SEO, App Router, edge middleware for auth guards
Why TypeScript	Catches bugs before runtime, forces correct data types, much better tooling
Why Tailwind	Utility-first CSS, zero runtime overhead, consistent dark mode, tiny bundle
SSR Benefit	Google can read pages for SEO; first paint is faster; auth redirect at edge

2.3 Database — PostgreSQL + Redis

Service	Role	Why This Choice
PostgreSQL	Permanent data	ACID-compliant, battle-tested, free, excellent for user and session data
asyncpg	DB driver	Async PostgreSQL driver — 3x faster than psycopg2 for concurrent requests

Service	Role	Why This Choice
PgBouncer	Pooler	1000 app connections share only 25 real DB connections — massive efficiency
Redis	Temp storage	Sub-millisecond OTP store, atomic rate-limit counters, session cache
hiredis	Redis driver	C extension parser — 10x faster Redis communication

3. System Architecture

3.1 Two-Repository Structure

Critical Concept

The backend and frontend are completely separate applications that communicate **ONLY** through API calls. The frontend knows nothing about the database. The backend knows nothing about the UI. This separation makes both easier to scale, test, secure, and maintain independently.

	backend-api	frontend-app
Language	Python 3.12	TypeScript / JavaScript
Framework	FastAPI	Next.js 14 App Router
Port	8000	3000
Knows DB?	Yes — queries directly	No — asks backend via API
Auth role	Issues JWT tokens in cookies	Sends credentials, reads cookies

3.2 OTP Login Flow — Step by Step

What happens behind the scenes when a user logs in with an email OTP:

1. User opens browser and navigates to `yoursite.com/login`
2. Next.js server renders the login page and sends HTML to browser
3. User enters email and clicks 'Send Code' button
4. Browser sends `POST /api/v1/auth/login/otp/email/request` to FastAPI
5. Nginx checks rate limit — max 5 OTP requests per 15 minutes per IP address
6. FastAPI rate limiter checks Redis sliding-window counter for this IP
7. FastAPI generates a cryptographically secure 6-digit OTP using Python secrets module
8. FastAPI creates an HMAC-SHA256 signature binding the OTP to the user's email address
9. FastAPI stores OTP + signature in Redis with automatic 10-minute expiry
10. FastAPI calls SendGrid API to send the OTP code to the user's inbox
11. FastAPI returns HTTP 202 Accepted — always, even for unknown emails (prevents enumeration)
12. User receives the email, copies the 6-digit code, and types it into the form
13. Browser sends `POST /api/v1/auth/login/otp/email/verify` with the submitted code
14. FastAPI validates HMAC signature and checks Redis for matching stored OTP
15. FastAPI immediately deletes the OTP from Redis — one-time use only, replay-proof
16. FastAPI queries PostgreSQL to look up the user account

17. FastAPI creates a JWT access token (15 minutes) and refresh token (7 days)
18. FastAPI sets BOTH tokens as HttpOnly + Secure + SameSite=Strict cookies
19. FastAPI writes the login event to the audit_logs table in PostgreSQL
20. Browser receives the cookies — JavaScript cannot read them, XSS-immune
21. User is redirected to the dashboard — session is active

3.3 Complete Directory Structure

File / Folder	What It Does
backend-api/	The entire Python FastAPI backend application
app/main.py	Entry point — starts server, registers routes, middleware
app/core/config.py	Reads .env and validates all environment variables at startup
app/core/security.py	All cryptography: JWT, bcrypt, OTP, HMAC, cookie helpers
app/core/rate_limiter.py	Redis sliding-window — blocks excessive requests per IP
app/api/v1/endpoints/auth.py	All auth routes: register, login, verify, refresh, logout
app/models/user.py	DB table definitions: User, UserSession, AuditLog
app/schemas/auth.py	Pydantic input/output shapes with strict validation
app/db/session.py	PostgreSQL async connection pool manager
app/db/redis.py	Redis connection pool manager
app/services/otp_service.py	Create, store, verify, delete OTPs in Redis
app/services/notification_service.py	Send emails via SendGrid, SMS via Twilio
app/services/error_notification_service.py	Build rich error reports and email support team
app/middleware/error_handler.py	Catches all errors, extracts context, notifies team
app/utils/ip_info.py	Geographic location lookup from IP address
alembic/	Database migration system — tracks schema changes
tests/	Automated test suite covering all auth flows
.env.example	Safe template — shows needed variables, no real secrets
.env	YOUR local secrets — NEVER committed to Git
Dockerfile	Packages the app into a portable Docker container
requirements.txt	List of all Python packages the backend needs

File / Folder	What It Does
frontend-app/	The entire Next.js TypeScript frontend
app/layout.tsx	Root layout wrapping all pages with AuthProvider
app/login/page.tsx	Login page with all four authentication methods
app/register/page.tsx	Registration form with full client-side validation
app/verify/email/page.tsx	Email verification — handles link click or manual OTP
app/verify/phone/page.tsx	Phone verification — enter the SMS OTP code
app/dashboard/page.tsx	SSR protected page — server fetches user before render
components/auth/AuthProvider.tsx	React context: user state, logout, token expiry
components/ui/	Reusable: InputField, SubmitButton, ErrorBanner
hooks/useTokenRefresh.ts	Silently refreshes access token 2 min before expiry
lib/api.ts	Typed API client — functions for all backend calls
types/auth.ts	TypeScript interfaces for all request/response shapes
middleware.ts	Edge middleware — redirects unauthenticated users
infra/nginx/dev.conf	Local development reverse proxy configuration
infra/nginx/prod.conf	Production: TLS, security headers, rate limits, gzip
infra/k8s/	Kubernetes deployment files for large-scale production
.github/workflows/ci.yml	CI/CD: lint, test, build Docker, deploy automatically
docker-compose.yml	Starts all services together with one command

4. Scale Strategy — Small Today, Large Tomorrow

The application starts small but the architecture supports massive growth. Here is exactly how infrastructure evolves as user numbers grow — without rewriting a single line of application code.

	Phase 1 — Start	Phase 2 — Growth	Phase 3 — Scale
Users	Up to 10,000	10,000 to 500,000	500,000+
Backend	Single VPS Docker	AWS ECS Fargate (2-8)	Kubernetes (3-50 pods)
Frontend	Docker on VPS	Vercel global edge	Vercel Enterprise
Database	Local PostgreSQL	AWS RDS Multi-AZ	Aurora Serverless v2
Redis	Local Redis	ElastiCache single	ElastiCache Cluster
Monthly cost	\$40 - \$80 USD	\$200 - \$500 USD	\$2,000+ USD
Code changes	None	None	None

Key Insight

The reason code never changes between phases is that FastAPI is stateless — no user data is stored in application memory. All state lives in PostgreSQL and Redis. Adding more containers simply means more capacity. This is the correct architectural pattern for scalable systems.

5. Document Series — Reading Order

Doc	Title	What You Will Learn
1	Project Overview (this document)	Architecture, tech stack choices, how all pieces connect
2	Environment Setup Guide	Installing software, creating databases, running the project
3	Security & Secrets Management	Environment variables, .env files, .gitignore, protecting production secrets
4	Deployment & Operations	Going live, CI/CD pipeline, monitoring, scaling, troubleshooting

Before You Start

Read this document completely before touching any code. Understanding the big picture first prevents 90% of beginner confusion. When something does not make sense later, return to the relevant section here before searching online.